COS 350 Lecture 8

bitwise operations

unix: tail

lseek

unix: sort

pointers to functions

qsort

Powers of 2:

decimal	binary
1	1
2	10
4	100
8	1000
16	10000
32	100000
64	1000000
128	10000000
256	100000000
512	1000000000

```
gcc accepts binary constants starting with 0b

int mask = 0b000111000;

This is not standard C.
```

Base 8 – octal – in C write octal numbers with a leading 0

decimal	binary	octal
1	1	01
2	10	02
4	100	04
8	1000	010
16	10000	020
32	100000	040
64	1000000	0100
128	10000000	0200
256	100000000	0400

Be careful with numbers starting with a 0

```
int zip = 04103;

printf("%05d", zip);   →   02115
```

Base 16 – hexadecimial – in C write with leading 0x

decimial	binary	octal	hexadecimal
1	1	01	0x1
2	10	02	0x2
4	100	04	0x4
8	1000	010	0x8
16	10000	020	0x10
32	100000	040	0x20
64	1000000	0100	0x40
128	10000000	0200	0x80
256	100000000	0400	0x100
512	1000000000	01000	0x200
1024	10000000000	02000	0x400

digits: 0 1 2 3 4 5 6 7 8 9 A B C D E F

A = 10, B = 11, C = 12, D = 13, E = 14, F = 15

In the hardware everything is binary

decimial	binary	octal	hexadecimal
1	1	01	0x1
2	10	02	0x2
4	100	04	0x4
8	1000	010	0x8
16	10000	020	0x10
32	100000	040	0x20
64	1000000	0100	0x40
128	10000000	0200	0x80

converting numbers:

```
75 (decimal) = 64 + 8 + 2 + 1
             = 0b1001011 (binary)
             = 0113      (octal)
             = 0x4B      (hex)
```

	short a =>	0000000010100101
	short b =>	0000000001001111
Bitwise operations:		
bitwise AND	a & b	0000000000000101
bitwise OR	a | b	0000000011101111
bitwise XOR	a ^ b	0000000011101010
1's complement	~a	1111111101011010
Bit shift operators:		
left shift	a << 3	0000010100101000
right shift	a >> 2	0000000000101001
right shift (signed)	~a >> 2	1111111111010110

	alpha	red	green	blue
Bitfield encoding:				
`int color = 0xffbc42f4;`	11111111	10111100	01000010	11110100
Masking:				
`int mask = 0x000000ff;`	00000000	00000000	00000000	11111111
Extracting a field:				
`blue = color & mask;`	00000000	00000000	00000000	11110100
`green = color>>8 & mask`	00000000	00000000	00000000	01000010
`red = color>>16 & mask;`	00000000	00000000	00000000	10111100
`alpha = color>>24 & mask;`	00000000	00000000	00000000	11111111
Setting bits:				
`color |= mask;`	11111111	10111100	01000010	11111111
Clearing bits:				
`color &= ~mask`	11111111	10111100	01000010	00000000

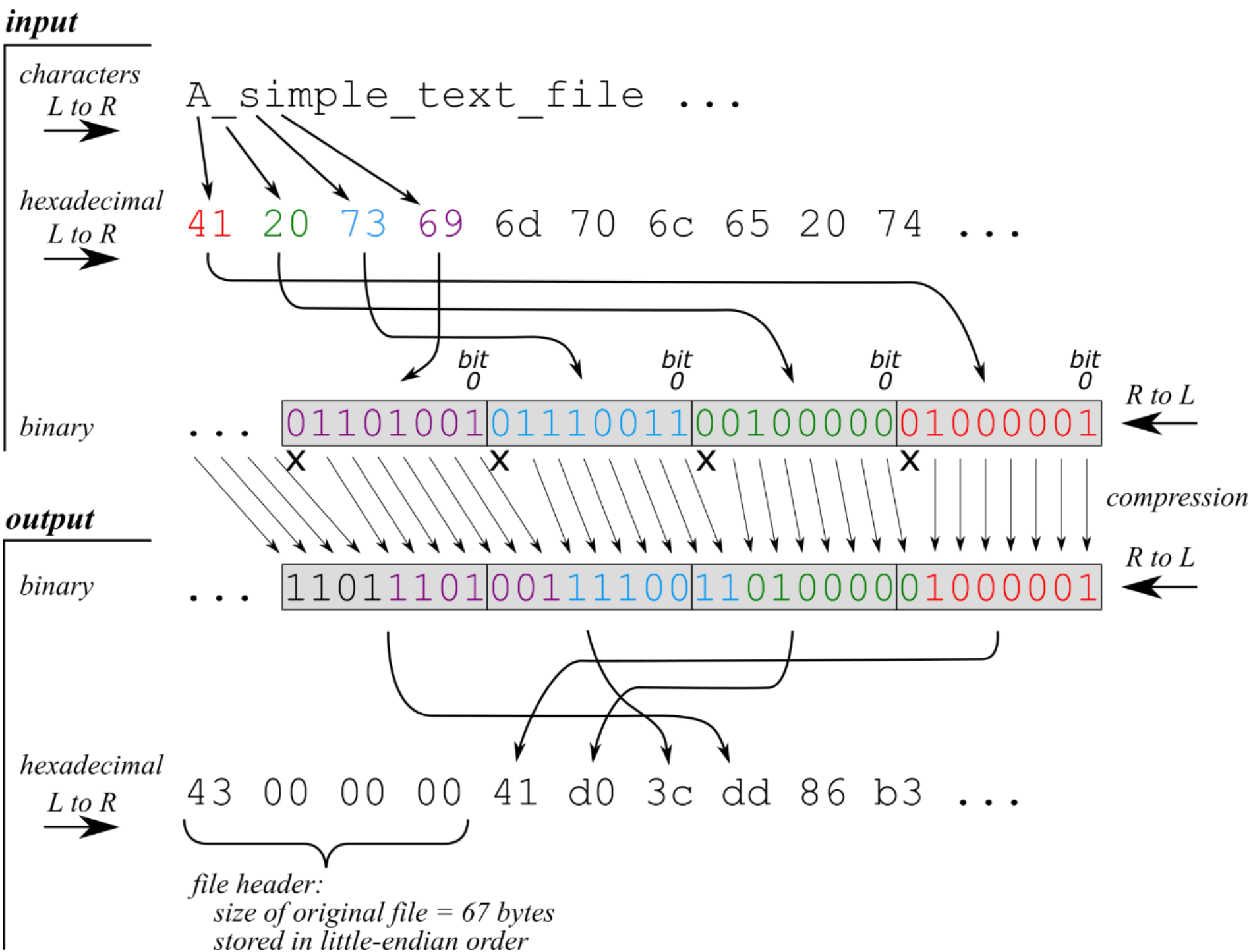
Numerical tricks with powers of 2 in binary:

`int a = 75;`	00000000000000000000000001001011
Remainder % 8:	
`a & 0x7`	00000000000000000000000000000111
`=> 3`	00000000000000000000000000000011
Round down multiple of 8:	
`a & ~0x7`	11111111111111111111111111111000
`=> 72`	00000000000000000000000001001000
Round up to multiple of 8:	
`(a+7) & ~0x7 => 82`	00000000000000000000000001010010
	11111111111111111111111111111000
`=> 80`	00000000000000000000000001010000
Multiply by 8:	
`a << 3 => 600`	00000000000000000000001001011000
Divide by 8:	
`a >> 3 => 9`	00000000000000000000000000001001

example using bit operations

```
unsigned int
endianSwap(unsigned int orig)
{
  unsigned int a,b,c,d,result;

  a = orig & 0xff000000;
  b = orig & 0x00ff0000;
  c = orig & 0x0000ff00;
  d = orig & 0x000000ff;

  a = a >> 24;
  b = b >> 8;
  c = c << 8;
  d = d << 24;

  result = a | b | c | d;

  return result;
}
```

```
orig = 0x12345678 =
00010010001101000101011001111000

00010010000000000000000000000000
00000000001101000000000000000000
00000000000000000101011000000000
00000000000000000000000001111000

00000000000000000000000000010010
00000000000000000011010000000000
00000000010101100000000000000000
01111000000000000000000000000000

01111000010101100011010000010010
= 0x78563412
```


input
characters
L to R
A_simple_text_file ...
hexadecimal
L to R
41 20 73 69 6d 70 6c 65 20 74 ...
bit 0
bit 0
bit 0
bit 0
R to L
binary
... 01101001 01110011 00100000 01000001
compression
output
binary
... 11011101 00111100 11010000 01000001
R to L
hexadecimal
L to R
43 00 00 00 41 d0 3c dd 86 b3 ...
file header:
size of original file = 67 bytes
stored in little-endian order

```
unix> tail words
zoology
zoom
Zorn
Zoroaster
Zoroastrian
zounds
z's
zucchini
Zurich
zygote
```

NAME

lseek - reposition read/write file offset

SYNOPSIS

```
#include <unistd.h>

off_t lseek(int fd, off_t offset, int whence);
```

DESCRIPTION

The lseek() function repositions the offset of the open file associated with the file descriptor fd to the argument offset according to the directive **whence**:

SEEK_SET = The offset is set to offset bytes.
SEEK_CUR = The offset is set to its current location plus offset bytes.
SEEK_END = The offset is set to the size of the file plus offset bytes.

RETURN VALUE

Upon successful completion, lseek() returns the resulting offset location as measured in bytes from the beginning of the file. Otherwise, a value of (off_t) -1 is returned and errno is set to indicate the error.

SORT(1) User Commands SORT(1)

NAME

sort - sort lines of text files

SYNOPSIS

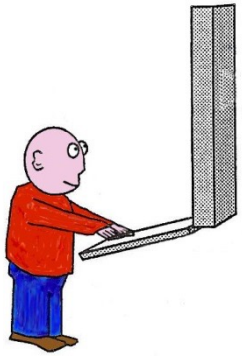
sort [OPTION]... [FILE]...

DESCRIPTION

Write sorted concatenation of all FILE(s) to standard output.

-n	compare according to string numerical value
-r	reverse the result of comparisons
-k f	start at field f (f starts at 1)
-k f.p	start at char position p in field f (p starts at 1)
-tc	use character c as the delimiter instead of whitespace

```
field:    | 1 | 2        | 3
          | an | example | line
position: | 12 | 12345678 | 123456
```



demo sort students

What do these three functions have in common?

```
int min(int a, int b)
{
    return a < b ? a : b;
}

int max(int a, int b)
{
    return a > b ? a : b;
}

int sum(int x, int y)
{
    return x+y;
}
```

you can have a pointer to a function:

```
int (*fn)(int, int);

fn = min;

x = fn(2, 6);      returns 2

fn = max;

y = fn(2,6);       returns 6

fn = sum;

z = fn(2,6);       returns 8
```

The main use of function pointers is to pass one function to another.

Example: a function that operates on an array

```
int a[20];
…
val1 = arrayop(a, 20, min);          returns the minimum in the array

val2 = arrayop(a, 20, max);          returns the maximum in the array

val3 = arrayop(a, 20, sum);          returns the sum of the array
```

```
// apply the function to the array
int arrayop(int a[], int n, int (*f)(int, int))
{
    int k, tmp;

    tmp = a[0];
    for (k = 1; k < n; k++)
        tmp = f(tmp, a[k]);

    return tmp;
}
```

NAME

qsort - sorts an array

SYNOPSIS

```
#include <stdlib.h>

void qsort(void *base, size_t nmemb, size_t size,
           int(*compar)(const void *, const void *));
```

DESCRIPTION

The qsort() function sorts an array with **nmemb** elements of size **size**. The **base** argument points to the start of the array.

The contents of the array are sorted in ascending order according to a comparison function pointed to by **compar**, which is called with two arguments that point to the objects being compared.

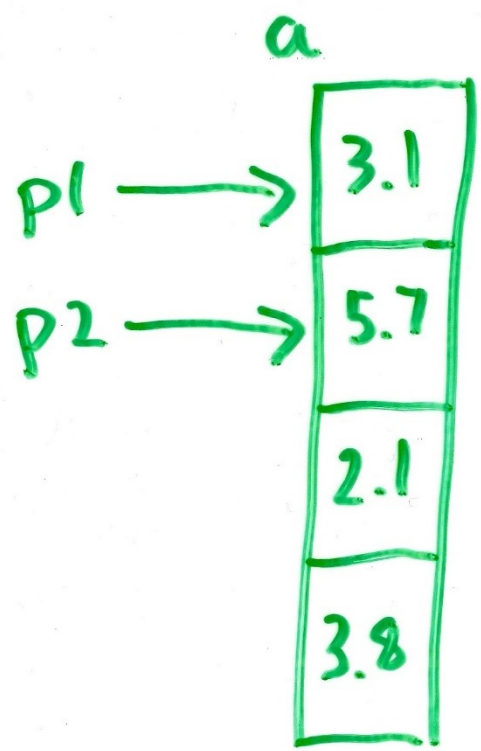
The comparison function must return an integer less than, equal to, or greater than zero if the first argument is considered to be respectively less than, equal to, or greater than the second. If two members compare as equal, their order in the sorted array is undefined.

```
void qsort(void *base, size_t nmemb, size_t size,
                int(*compar)(const void *, const void *));
```

---

example use:

```
double a[4] = {3.1, 5.7, 2.1, 3.8};

//    base n   width                cmp func
qsort( a,  4,  sizeof(double),  dbl_cmp_fn );
```



example compare function:

```
// compare function for qsort of array of doubles
int dbl_cmp_fn(const void *p1, const void *p2)
{
    const double d1 = *((double*)p1);
    const double d2 = *((double*)p2);

    if (d1 < d2)   return -1;
    if (d1 == d2) return 0;
    else              return 1;
}
```

Our main use of function pointers will be for OS signal handlers.

defines a new type name: sighandler_t that is:
a pointer to a function
that takes one integer argument
and returns void

```
typedef  void  (*sighandler_t)(int);

sighandler_t  signal(int  signum,  sighandler_t  handler);
```

signal is a function with 2 arguments
(a signal number & a pointer to a signal handling function)
and returns a pointer to the old signal handling function